If none of the clauses evaluate truthy, you'll receive a CondClauseError. Add true -> as the final clause for a fallback that guarantees a match.

Let's switch gears to see how pattern matching affects functions.

Level Up Your Functions

Pattern matching takes function overloading to a new level. The most basic type of function overloading lets you define the same function name with a different number of arguments, but Elixir supports more than this.

We'll go over function overloading, how it's implemented, and how it can be used to create seamless recursive functions. But first, we need to cover a helpful addition to pattern matching called guards.

Guards

While pattern matching is really powerful, it can't do certain things. Pattern matching operates on the structure of a value, but sometimes we want to go beyond that. For example, we may want to know whether a number is within a certain range, whether a value is in a predefined list, or what the type of a value is. Guards let us do all of these things.

Guards are statements that use functions or operators to add additional checks to a pattern match. They can be used anywhere that pattern matching is used: case and with statements, comprehensions, and function definitions. Plus, you can write your own guards—as long as they follow limitations enforced by Elixir's compiler. But that's not common, so we won't cover it in this book.

Let's start with a simple example. We'll use a guard to check whether an integer is within a particular range. The guards appear after when in each line:

```
$ iex
iex> number = 7
case number do
  n when n < 5 -> :low
  n when n >= 5 and n < 10 -> :medium
  n when is_number(n) -> :high
end

:medium
```

Try this example out with different values for number. Use 0, 10, and "string". Let's consider what happens if you put "string" into this case statement. We get a CaseClauseError, despite it matching n in every single clause. The guards that appear after when cause the value to match nothing.